Maze Solving: BFS vs A* vs GBFS

Nicolas Stamper, Elliot Lindsey

Overview

A Maze Solving problem was chosen for this project, involving finding the shortest path possible from a starting location to an ending location within a graph of vertices with weighted edges to represent maze path lengths. The three algorithms we chose to analyze against this problem are BFS, a brute force algorithm that guarantees finding the shortest possible solution, A*, a heuristic-based function, and Greedy Best First Search (GBFS), a nature-inspired algorithm that uses similar mechanics to A*. BFS works by exploring all possible paths from the starting position, maintaining equal length of the paths until it discovers a solution, A* uses estimates of the distance remaining to the goal position to judge which paths are more likely to be an efficient solution, and GBFS attempts to find paths with the most potential progress and least estimated cost without precisely prioritizing length.

Objective and Research Questions

This experiment empirically compares BFS, A*, and GBFS across four maze sizes to evaluate how each algorithm scales in terms of runtime, nodes explored, path quality, and reliability. We investigated the following research questions prior to running the experiment:

1. How does the runtime of A* scale as maze size increases, and does it remain faster than BFS at all sizes?
2. At which maze size, if any, does GBFS begin to clearly outperform both BFS and A* in terms of runtime?
3. How do the path lengths produced by GBFS compare to the optimal paths produced by BFS and A*, and does the gap widen with maze size?
4. How do the success rates of the three algorithms change as maze size increases, and which algorithm is most reliable across all sizes?

Prior to the experiment, we expected BFS to be the slowest but reliable, A* to balance speed and optimality, and GBFS to be the fastest but sacrificing path quality and failing on smaller mazes.

Experiment

We chose to use Python to implement the algorithms since we had a fair amount of familiarity with the language going into the project. For the input data, we used synthetically generated square mazes with randomized obstacle placement, keeping the obstacle frequency constant to fix the difficulty of the input mazes. For each algorithm, we measured its average time, nodes visited, path length, and success rate for 4 batches of 20 mazes. The dimensions of the first

batch were 10x10, the second batch was 20x20, the third batch was 40x40, and the fourth batch was 80x80, strictly following a geometric progression of input size.

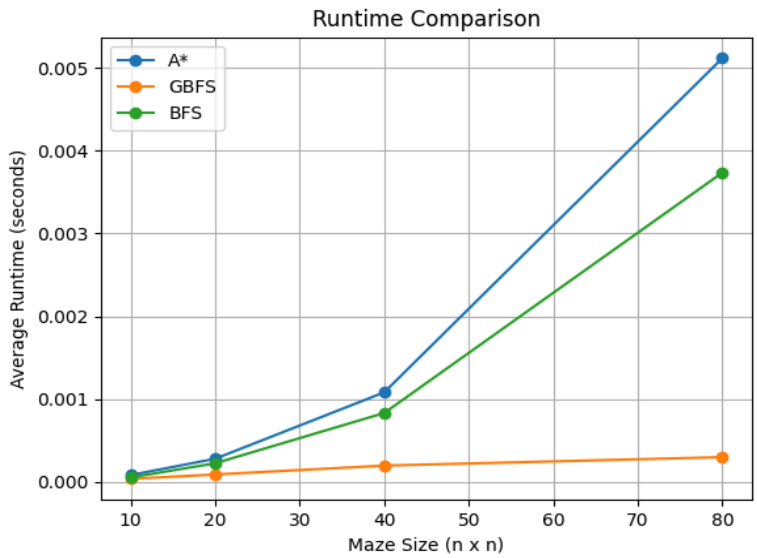
Experiments were run on a Windows 10 (64-bit) machine equipped with an AMD Ryzen 9 5900X 12-Core Processor, 64 GB of RAM, an NVIDIA GeForce GTX 1660 SUPER GPU, and a 954 GB SSD. We found BFS useful for reference comparisons such as success rates and optimal path length, since it guarantees finding the shortest path in any solvable maze and serves as the most straightforward baseline for the other algorithms to optimize on.

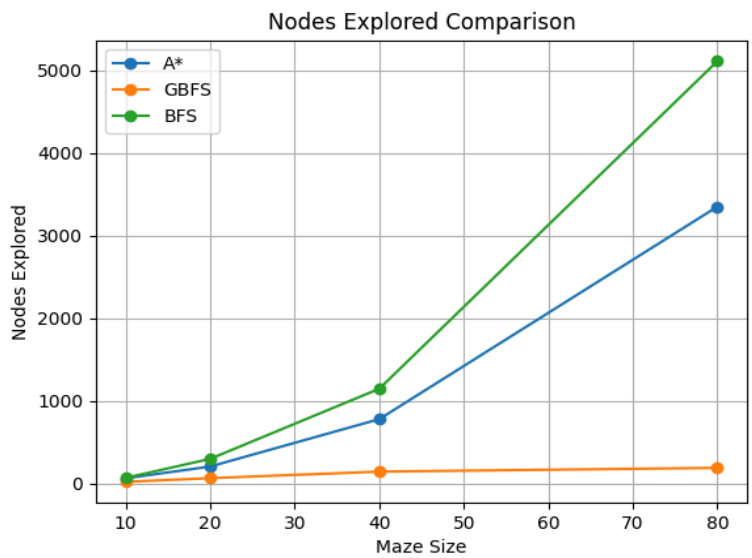
Results

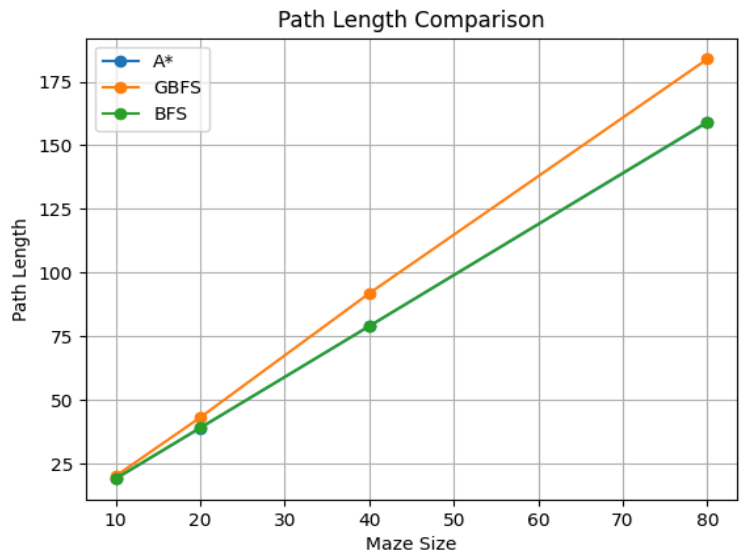
Summary of Average Results Across All Maze Sizes

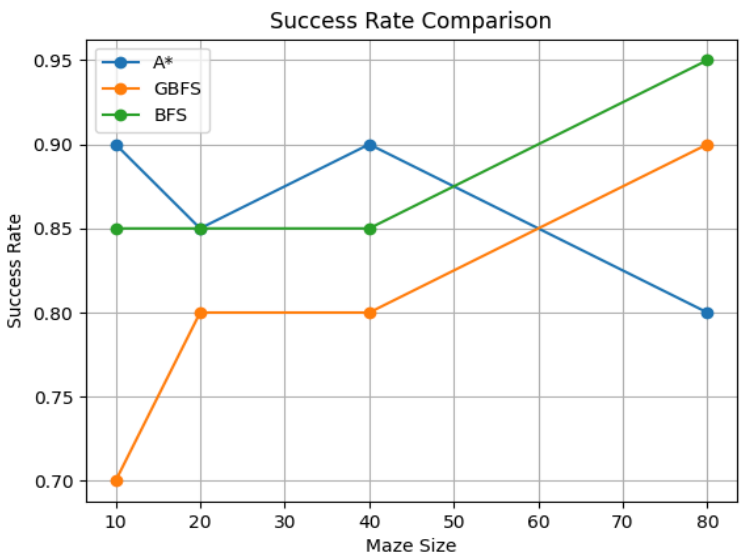
Algorithm	Maze Size	Avg Time (s)	Avg Nodes	Avg Path Length	Success Rate
A*	10x10	0.0000830	66.75	19.00	0.90
	20x20	0.000283	209.25	39.00	0.85
	40x40	0.00116	782.50	79.00	0.90
	80x80	0.00574	3353.85	159.00	0.80
GBFS	10x10	0.0000414	24.00	20.14	0.70
	20x20	0.0000950	68.70	43.13	0.80
	40x40	0.000218	147.70	91.88	0.80
	80x80	0.000315	193.25	183.89	0.90
BFS	10x10	0.0000584	71.85	19.24	0.85
	20x20	0.000236	301.95	39.12	0.85
	40x40	0.000871	1149.95	79.00	0.85
	80x80	0.00392	5113.90	159.21	0.95

Performance Graphs









Results and Limitations

Runtime: GBFS had lower average runtimes than both A* and BFS at every size tested. At the 80x80 size, GBFS averaged 0.000315 seconds compared to 0.00574 seconds for A* and 0.00392 seconds for BFS. Both A* and BFS exhibited linear growth in runtime as input size doubles, consistent with their $\Theta(V + E)$ worst-case complexity. GBFS scales nearly flat, reflecting its aggressive heuristic pruning.

Nodes Explored: BFS explored the most nodes at every size, reaching an average of 5,113.9 nodes at 80x80, which is consistent with its brute-force exhaustive expansion. A* explored significantly fewer nodes, 3,353.85 at 80x80, by using the $f(n) = g(n) + h(n)$ evaluation to prune bad paths. GBFS explored by far the fewest nodes, 193.25 at 80x80, confirming that evaluating only $h(n)$ produces aggressive but short-sighted search behavior. The node counts for BFS and

A* both grow approximately in proportion to the maze area, while GBFS's count grows very slowly, as it consistently finds and follows promising corridors without broad exploration.

Path Length: BFS and A* produced nearly identical path lengths at all sizes, reflecting BFS's guaranteed optimality and A*'s use of an acceptable Manhattan distance heuristic. GBFS produced noticeably longer paths, especially at larger maze sizes, as at 80x80, GBFS averaged a path length of 183.89 versus 159.00 for both A* and BFS. This confirms the expectation that GBFS trades solution quality for speed.

Success Rate: GBFS had the lowest success rate at 10x10, being 0.70, likely because small, constrained mazes are the weakness of a pure heuristic search, GBFS can easily get trapped in dead ends with insufficient backtracking capabilities. BFS was the most consistently reliable, finishing with the highest success rate of 0.95 at 80x80, which is expected given its complete and optimal nature. A*'s success rate declined slightly at larger sizes, 0.80 at 80x80, which may reflect edge cases in the maze generation producing configurations where the heuristic underestimated costs near walls. Overall, success rates did not follow a simple trend for any algorithm, suggesting variability in individual maze difficulty despite the fixed obstacle frequency.

Limitations: Several limitations constrained the conclusions of this experiment. First, all mazes were square with a fixed obstacle frequency, testing varying aspect ratios or obstacle densities would offer a more complete picture of algorithm robustness. Second, the sample size of 20 trials per configuration is small, and the variability in success rates across sizes suggests that a larger sample would produce more stable estimates. Third, memory usage was not formally measured. Fourth, the exclusion of Ant Colony Optimization (ACO) due to a team member departure and high costs means the nature-inspired algorithm category is represented only by GBFS, which, while heuristically motivated, is arguably less distinctly inspired than ACO. If this experiment were repeated, we would increase the number of trials to at least 50, measure memory usage, and explore a range of obstacle densities to better characterize each algorithm's failure conditions.

Sources

Source	Use
Claude & ChatGPT	Creating and refining code used in the program, helped update the Planning Stage checkpoint and some inspiration for the PowerPoint presentation.
GeeksforGeeks.org & Wikipedia.org	Obtaining information and researching the three algorithms of BFS, A*, and GBFS, and finding out how they work in maze solving.

Contributions

Stage	Nicolas	Elliot
Brainstorming	Researched A* information	First checkpoint document layout, initial brainstorming, DFS information

Stage	Nicolas	Elliot
Coding	Wrote Python scripts for each algorithm and executable script	Examined the Python code for any faults
Experiment Design	Implemented input size structure and trial size, collected results	Suggested input size structure and input data types
Visualization / Results	Recorded all results and formed all graphs	Examined the results for any faults or potential inconsistencies
Report Writing	Created and outlined report, worked on filling it out	Worked on filling it out alongside Nicolas
Presentation	Created and outlined slides, inserted graphs	Worked on basic art, formatting, and organization
Communication	Communicated work that had been done and what needed to get done	Initiated communication and arranged meetings
Submission	Created the GitHub repository and added content that would be submitted	Submitted the GitHub repository for the second checkpoint

Main Difficulties

Trying to understand and pick out information from long outputs that GenAI gave, as well as minor imprecisions in the given prompts, was an inconvenience but nothing that greatly hindered progress. Defining which features of the code were specific to each algorithm versus generically shared across all three was challenging, though it was eventually sorted out into a clean modular structure. Additionally, having a member drop out of the group late into the semester was rough, especially since they were the one who would have been working on the former nature-inspired algorithm, Ant Colony Optimization (ACO). In response, ACO was replaced with a more feasible algorithm, Greedy Best-First Search (GBFS).